Journal of Science and Technology – Smart Systems and Devices
Volume x, Issue x, Month Year, ppp-ppp

Adaptive Multi-Path Inference for Multi-Tenant LLM Systems with Shared Models and LoRA-based Personalization

Tran Bui Tung Linh[1], Hoang Trung Kien[2], Nguyen Thanh Trung[3]
Faculty of Mathematics and Informatics, Hanoi University of Science and Technology, Ha Noi, Vietnam
**Corresponding author email: linh.tbt227057@hust.edu.vn*

Abstract

Enterprise LLM applications are increasingly being utilized in multi-tenant settings, specifically where heterogeneous queries from different organizations are funneled into shared infrastructures, while under a mandate to ensure both efficiency, isolation, and tenant-specific behavior. In such environments, single-path inference pipelines often prove insufficient, as simple and complex requests are forced through the same expensive execution process. This paper presents an adaptive multi-path inference architecture for multi-tenant LLM systems, capable of dynamically selecting among retrieval-based generation, tool-based processing for structured or semi-structured data, and direct generation according to query characteristics. The system is built around a shared base model to elevate resource efficiency, while tenant-specific Low-Rank Adaptation (LoRA) modules support lightweight tenant-level customization without replicating the full model for each tenant. To support practical deployment, we further introduce a tenant-aware runtime with isolated knowledge stores, isolated conversational memory, and dynamic adapter loading to reduce memory overhead during inference. The proposed design positions retrieval and tools as execution paths within a broader serving architecture rather than as the defining identity of the system. Experimental evaluation in a multi-tenant setting shows that the architecture improves latency and memory efficiency over fixed single-path baselines while maintaining competitive response quality. These results suggest that adaptive multi-path inference, combined with shared-model serving and LoRA-based personalization, is a promising design paradigm for scalable and cost-efficient enterprise LLM systems.

Keywords: Adaptive inference, LoRA personalization, Multi-tenant LLM systems, Resource efficiency.

1. Introduction

1.1. Background and Motivation

Large language models (LLMs) are surging in adoption, assuming the role of enterprise assistants, document intelligence, customer support, and internal knowledge systems. In practice, these deployments rarely serve a single homogeneous workload. Instead, they must process heterogeneous requests ranging from general conversational questions to document-grounded queries and structured data operations. This heterogeneity becomes more challenging in multi-tenant environments, where multiple companies and organizations share the same serving infrastructure but still expect tenant-specific behavior, data isolation, and predictable performance.

Many practical deployments still rely solely on a fixed inference pipeline for all requests. Although such a design is simple to implement, it is inefficient for enterprise workloads as different query types require different execution flows. Routing every request through the same expensive retrieval or generation process can lower response rates, waste compute resources, and blur the system-level trade-off between answer quality and serving efficiency.

1.2. Research Gaps in Multi-tenant LLM Serving

The first gap is an *efficiency gap*. Existing one-path-fits-all pipelines do not distinguish well between simple queries, knowledge-grounded queries, and structured data queries. As a result, lightweight requests may still trigger retrieval, long-context prompting, or unnecessary model calls.

The second gap is a *personalization gap*. Multi-tenant applications often require tenant-specific language style, policies, and domain preferences. A straightforward solution is to maintain separate models for different tenants, but this quickly becomes impractical because full-model duplication increases memory cost and complicates system maintenance.

The third gap is a *multi-tenant systems gap*. A realistic serving system must jointly support tenant isolation, scalable resource sharing, and personalization under the same runtime. Many existing approaches optimize only one

p-ISSN 3093-3285
e-ISSN 3093-3315
https://doi.org/10.51316/jst.xxx.ssad.xxxx.xx.x.x
Received: ...../..../.....; Revised: ...../..../.....;
Accepted: ...../..../.....; Online: ...../..../.....

dimension, such as retrieval quality, adapter efficiency, or serving throughput, without addressing the interaction among routing, shared serving, and tenant-aware isolation in a unified architecture.

1.3. Adaptive Multi-Path Serving Architecture

To address these gaps, this paper frames the problem as adaptive multi-path serving for multi-tenant LLM systems. Rather than treating retrieval-augmented generation as the default identity of the system, we treat retrieval as one execution path among several possible paths. The proposed architecture selects between tool-based processing, retrieval-based generation, and direct generation based on query characteristics and tenant context.

The system adopts a shared base model as the common serving backbone, which improves scalability and reduces memory usage compared with per-tenant model replication. To preserve tenant-specific behavior under shared serving constraints, we incorporate Parameter-Efficient Fine-Tuning (PEFT) through Low-Rank Adaptation (LoRA). In this design, LoRA is not merely an auxiliary personalization module; it is the practical mechanism that resolves the trade-off between shared-model efficiency and tenant-level customization.

The runtime is tenant-aware by construction. Each tenant is associated with isolated knowledge storage, isolated conversational memory, and an optional personalization adapter. Tenant-specific LoRA adapters are loaded dynamically during inference so that the system can support customization without keeping all adapters resident in memory at the same time.

1.4. Scope and Contributions

The paper is positioned as a systems-and-application study on multi-tenant LLM serving. Its central claim is not that retrieval, tools, or LoRA are individually new, but that their coordinated integration yields a practical serving architecture for heterogeneous enterprise workloads.

1.5. Contributions

The main contributions of this work are summarized as follows:

- We propose a unified adaptive multi-path serving architecture for heterogeneous multi-tenant LLM workloads, where queries are routed among tool, retrieval, and direct-generation paths.
- We integrate shared-model serving with tenant-specific LoRA personalization to balance resource efficiency and customization without full-model duplication.
- We introduce a tenant-aware runtime that combines isolated knowledge storage, isolated memory, and dynamic adapter loading for scalable multi-tenant deployment.
- We evaluate the architecture using system-oriented metrics, including latency, memory usage, and response quality, to compare adaptive serving against fixed single-path baselines.

2. System Architecture

2.1. Overall Structure

The proposed system is designed as a modular multi-tenant serving architecture that shares the core model across tenants while preserving tenant-specific data boundaries. Its main components are the data ingestion layer, the tenant-aware storage and retrieval layer, the adaptive inference engine, and the API serving layer.

At the system level, the architecture is not organized around a single fixed RAG pipeline. Instead, it uses adaptive multi-path inference, where retrieval is one possible path inside a broader serving framework. A shared base language model is deployed as the main inference engine and serves multiple tenants through a common runtime.

Each tenant maintains an isolated knowledge base and conversation state. During query processing, the runtime first identifies the tenant context, then applies route selection, and finally invokes the most appropriate execution path under that tenant scope.

The routing layer determines whether a query should be processed through tool-based execution, retrieval-based generation, or direct generation. This design reduces unnecessary LLM calls and better matches execution cost to query complexity.

2.2. Data Ingestion and Knowledge Base Construction

The data ingestion layer is responsible for collecting, processing, and indexing tenant-specific data from multiple sources.

Supported data types include:

- Unstructured documents (PDF, DOCX, HTML)
- Structured data (Excel, CSV)
- Web-based content (links, HTML pages)

The ingestion pipeline performs document parsing, text normalization, chunking, embedding generation, and vector storage. Each tenant's data is stored in an isolated partition to enforce separation and reduce cross-tenant leakage risk.

2.3. Retrieval Module

The retrieval module is responsible for fetching relevant context only when the selected route requires external grounding.

Given a user query and tenant identifier:

1) The query is converted into an embedding vector
2) Similarity search is performed within the tenant-specific index
3) Top-k relevant chunks are returned

To improve retrieval quality, the system can incorporate chunk filtering, re-ranking, and hybrid retrieval strategies. In the proposed architecture, retrieval is therefore a selective component of the serving process rather than the default path for every request.

2.4. Dynamic Inference Engine

The dynamic inference engine is the core component of the system. It combines a shared base LLM, a route-selection mechanism, and optional tenant-specific personalization modules.

2.4.1. Shared Base Model

A single base LLM is shared across all tenants, which reduces memory usage and simplifies deployment compared with maintaining a separate full model for each tenant.

2.4.2. Query Routing Mechanism

The system introduces a routing mechanism that maps incoming queries to one of several execution paths:

- Tool-based queries (e.g., Excel parsing, file inspection)
- Retrieval-based queries (RAG)
- General queries handled directly by the LLM

This routing structure minimizes unnecessary model invocations and improves latency for heterogeneous workloads.

2.4.3. Dynamic Adapter Loading

To support tenant-level customization, the system uses PEFT-based adapters, specifically LoRA, that are dynamically loaded during inference. Instead of keeping all adapters in memory, the runtime loads the required adapter on demand and releases it after use, which lowers memory overhead while preserving personalization capability.

2.5. API and Serving Layer

The architecture is exposed through a RESTful API implemented with FastAPI.

Key responsibilities include:

- Handling user requests
- Resolving tenant context
- Invoking the selected execution path
- Returning responses and associated provenance when applicable

The API layer also supports logging, monitoring, and integration with front-end applications. These functions are important for reproducibility and operational visibility in multi-tenant deployments.

3. Implementation Details

3.1. System Components

The system is implemented using a modular architecture, where each component is responsible for a specific function in the query processing pipeline.

The main components include:

- **Router module:** Determines the execution path for each query.
- **Workflow manager:** Orchestrates the end-to-end processing pipeline.
- **Retrieval module:** Handles tenant-scoped vector-based similarity search.
- **LLM service:** Manages inference using the shared base model.
- **Memory store:** Maintains tenant-specific data and context.
- **Tool module:** Processes structured data queries (e.g., Excel, CSV).

This modular design improves flexibility, scalability, and maintainability of the system.

3.2. Multi-tenant Design

Each tenant in the system is associated with:

- A unique tenant identifier
- An isolated knowledge base
- An isolated memory context
- An optional LoRA personalization adapter

The system ensures strict isolation between tenants by:

- Separating vector database partitions
- Maintaining independent memory contexts
- Routing queries based on tenant identifiers

At the same time, all tenants share a single base model, enabling efficient resource utilization.

3.3. Routing Optimization

The routing mechanism is designed to reduce unnecessary computational overhead and to better align execution cost with query requirements.

Instead of processing all queries through the LLM, the system dynamically selects the most efficient execution path:

- **Tool-based processing:** For structured data queries such as Excel or CSV
- **Retrieval-based generation:** For document-based queries (e.g., PDF)
- **Direct LLM inference:** For general queries

This approach reduces inference latency and token usage by avoiding a uniform pipeline for all requests.

3.4. Resource Optimization Strategy

The system incorporates multiple techniques to optimize resource usage:

- **Shared model architecture:** A single base LLM serves all tenants
- **Dynamic adapter loading:** Tenant-specific LoRA adapters are loaded on demand
- **Selective LLM invocation:** Retrieval and full generation are used only when justified by the route decision

These strategies reduce memory footprint and improve scalability in cloud environments.

4. Experimental Evaluation

4.1. Experimental Setup

The system is evaluated using a test set that covers heterogeneous enterprise-style queries:

- Structured data queries handled by the tool path
- Document-grounded queries handled by the retrieval path
- General reasoning queries handled by direct generation
- Multi-tenant requests executed under isolated tenant contexts

Two configurations are compared:

- Fixed single-path baselines, where queries are forced through one dominant processing pipeline
- The proposed adaptive system, which combines route selection, shared-model serving, and tenant-aware execution

4.2. Evaluation Metrics

The following metrics are used:

- Latency (ms): time taken to produce a final response
- Memory usage (MB): RAM or GPU consumption during serving
- Response quality: correctness and usefulness of generated answers
- Path suitability: whether the selected execution path matches the query type

4.3. Results and Analysis

The results show that adaptive multi-path inference reduces latency by avoiding unnecessary retrieval or generation steps for lightweight queries. The shared-model design also lowers memory usage compared with configurations that replicate full models or overuse retrieval-heavy execution.

Structured queries benefit the most from routing because they can be resolved through tools without incurring the full cost of long-context LLM inference. Retrieval-heavy queries still benefit from grounding when needed, while general queries avoid unnecessary external lookup.

4.4. Discussion

Overall, the evaluation supports the central systems claim of the paper: adaptive route selection, shared-model serving, and lightweight personalization work together to improve efficiency while preserving multi-tenant practicality.

5. Conclusion

This paper presents an adaptive multi-path serving architecture for multi-tenant LLM systems with shared models and LoRA-based personalization. The proposed design addresses three coupled requirements of enterprise deployment: efficient serving under heterogeneous workloads, tenant-specific customization without full-model duplication, and tenant-aware isolation within a shared runtime.

By treating retrieval, tool use, and direct generation as coordinated execution paths instead of forcing all requests through a single pipeline, the system better aligns inference cost with query needs. The shared base model improves resource efficiency, while dynamic LoRA loading enables lightweight personalization that remains compatible with multi-tenant deployment constraints.

5.1. Future Work

Future work includes:

- learning-based routing policies with calibrated confidence scores,
- stronger evaluation of personalization under tenant-specific benchmarks,
- larger-scale system experiments across more tenants and workloads, and
- deeper analysis of throughput, adapter switching overhead, and isolation guarantees in production-like settings.